

I. Introduction.....	1
II. C Program Structure and Examples	1
A. Basic Hello World Program.....	1
B. Program with User Input.....	1
C. Program with Formatted Output	2
III. GCC Compilation Process	2
A. Installing GCC on Linux.....	2
B. Four Stages of Compilation	3
C. Compilation Stage Examples	3
1) Stage 1: Preprocessing	3
2) Stage 2: Compilation to Assembly.....	3
3) Stage 3: Assembly to Object File.....	3
4) Stage 4: Linking	4
IV. Return Values and Exit Status	4
A. Understanding Return Values	4
B. Checking Exit Status	4
V. Knowledge Check Questions	5
A. Question 1	5
B. Question 2	5
C. Question 3	5
D. Question 4.....	5
E. Question 5	6
F. Question 6.....	6
VI. Lab Tasks Completed	6
1) Task 1	6
2) Task 2	6
3) Task 3	6
4) Task 4	6
5) Task 5	7
VII. Screenshots and Evidence	7
VIII. Conclusion.....	7
GitHub Repository	7

Lab 2: C Programming Basics

Lab Report and Knowledge Check

Course: ST5003CMD – Operating Systems, Security and Networks

Name: _____ Date: _____

I. Introduction

This report documents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The focus is on understanding C program structure, compilation stages, and basic Linux commands for software development.

II. C Program Structure and Examples

A. Basic Hello World Program

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Listing 1. Basic Hello World Program

[Insert screenshot here]

Alt text: Terminal showing hello.c compiled and run, printing "Hello, World!"

B. Program with User Input

This example demonstrates the use of printf and scanf for input/output operations:

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Listing 2. Program with User Input

[Insert screenshot here]

Alt text: Terminal showing the program running and asking for/printing an entered number

C. Program with Formatted Output

Demonstrating multiple format specifiers and variables:

```
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Listing 3. Program with Formatted Output

[Insert screenshot here]

Alt text: Terminal showing name, age, and height entered and printed back in the --- User Info --- block

III. GCC Compilation Process

A. Installing GCC on Linux

The following commands are used to install and verify GCC:

```
# Install build-essential package
sudo apt install build-essential

# Check GCC version
gcc --version
```

Listing 4. GCC Installation Commands

[Insert screenshot here]

Alt text: Terminal showing build-essential installing successfully and gcc --version output

B. Four Stages of Compilation

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

Stage	Command	Output	Purpose
1. Preprocessing	<code>gcc -E hello.c -o hello.i</code>	<code>.i</code> file	Expand headers & macros
2. Compiling	<code>gcc -S hello.i -o hello.s</code>	<code>.s</code> file	Generate Assembly code
3. Assembling	<code>gcc -c hello.s -o hello.o</code>	<code>.o</code> file	Create object file
4. Linking	<code>gcc hello.o -o hello</code>	executable	Final executable

Table I. The Four Stages of C Compilation

C. Compilation Stage Examples

1) Stage 1: Preprocessing

```
# Expand all headers and macros
gcc -E hello.c -o hello.i

# View the preprocessed output
cat hello.i | less
```

Listing 5. Preprocessing Stage

[Insert screenshot here]

Alt text: Terminal showing hello.i generated and its expanded contents viewed with cat/less

2) Stage 2: Compilation to Assembly

```
# Generate assembly code
gcc -S hello.i -o hello.s

# View assembly code
cat hello.s
```

Listing 6. Compilation Stage

[Insert screenshot here]

Alt text: Terminal showing hello.s generated and its assembly code contents

3) Stage 3: Assembly to Object File

```
# Create object file (machine code)
gcc -c hello.s -o hello.o

# Check file type
file hello.o
```

Listing 7. Assembly Stage

[Insert screenshot here]

Alt text: Terminal showing hello.o created and the file command confirming it is an ELF object file

4) Stage 4: Linking

```
# Create final executable
gcc hello.o -o hello

# Run the program
./hello
```

Listing 8. Linking Stage

[Insert screenshot here]

Alt text: Terminal showing the final hello executable created and run, printing its output

IV. Return Values and Exit Status

A. Understanding Return Values

The return value of `main()` indicates program success or failure to the OS:

```
#include <stdio.h>

int main() {
    printf("Program executed successfully\n");
    return 0; // 0 indicates success
}
```

Listing 9. Program with Return Value

[Insert screenshot here]

Alt text: Terminal showing the program printing its success message

B. Checking Exit Status

```
# Run program
./hello

# Check exit status (returns 0 for success)
echo $?

# Try with return 1
```

```
# Modify program to return 1, recompile, and check:
echo $? # Should return 1 (indicating error)
```

Listing 10. Checking Program Exit Status

[Insert screenshot here]

Alt text: Terminal showing echo \$? returning 0 after a successful run, then returning 1 after modifying the program

V. Knowledge Check Questions

The following questions assess understanding of Lecture 2 concepts.

A. Question 1

Q: What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

A: The `#include` directive tells the preprocessor to copy the entire contents of a specified header file into the source file before compilation starts. It gives a program access to functions, macros, and declarations defined elsewhere, so they do not need to be rewritten. System headers are included in angle brackets, while user-created headers use quotes. For example, `#include <stdio.h>` provides access to input/output functions such as `printf()` and `scanf()`.

B. Question 2

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

A: C compilation happens in four sequential stages, each moving the code closer to a runnable program. Preprocessing expands headers and macros, producing a `.i` file. Compiling translates that preprocessed code into assembly, producing a `.s` file. Assembling converts the assembly into machine code, producing a `.o` object file, which is finally linked with any needed libraries into the finished executable with no file extension.

C. Question 3

Q: What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

A: A return value of 0 tells the operating system that the program ran and finished successfully with no errors. A non-zero return value signals that something went wrong during execution, and different numbers can represent different specific error conditions. This value becomes the program's exit status, which can be checked immediately afterward using the shell command `echo $?`.

D. Question 4

Q: Write a C program that declares an integer variable, asks the user to input a value, and displays the entered value back to the user.

```
#include <stdio.h>

int main() {
    int value;

    printf("Enter a value: ");
    scanf("%d", &value);

    printf("You entered: %d\n", value);
}
```

```
    return 0;
}
```

A: The program declares an integer variable, prompts the user with `printf()`, and reads their input using `scanf()` with the `&` operator to store it directly into that variable's memory address. It then prints the stored value back out using another `printf()` with the `%d` format specifier.

[Insert screenshot here]

Alt text: Terminal showing the program prompting for a value, the user typing one, and it being echoed back

E. Question 5

Q: What GCC command would you use to: (1) only preprocess a file without compiling, (2) generate assembly code but not assemble it, (3) create an object file without linking?

A: To only preprocess, use `gcc -E hello.c -o hello.i`, which just expands headers and macros. To stop after generating assembly, use `gcc -S hello.i -o hello.s`. To create an object file without linking it into a full executable, use `gcc -c hello.s -o hello.o`. Each flag (`-E`, `-S`, `-c`) tells GCC to stop the pipeline at that specific stage.

F. Question 6

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

A: C is called a compiled language because its entire source code is translated into machine code by a compiler before the program ever runs, producing a standalone executable file. Interpreted languages, by contrast, are read and executed line-by-line at runtime by an interpreter, without ever producing a separate executable file. Compiled programs typically run faster since translation happens once ahead of time, while interpreted programs run slower but are often quicker to test and debug. Python and JavaScript are common examples of interpreted languages.

VI. Lab Tasks Completed

1) Task 1

Compiled `hello.c` with `return 0` and verified exit status using `echo $?`.

[Insert screenshot here]

Alt text: Terminal showing `hello.c` compiled, run, and `echo $?` returning 0

2) Task 2

Modified the program to return 1 and observed a different exit status.

[Insert screenshot here]

Alt text: Terminal showing the modified program run and `echo $?` returning 1

3) Task 3

Created a program accepting user input and displaying it back.

[Insert screenshot here]

Alt text: Terminal showing the input program running with a real value entered and displayed

4) Task 4

Executed all four compilation stages separately and documented the output files at each step.

[Insert screenshot here]

Alt text: Terminal or file listing (ls -l) showing hello.i, hello.s, hello.o, and hello all present

5) Task 5

Compiled and ran a program returning the integer value 50.

[Insert screenshot here]

Alt text: Terminal showing the program run and echo \$? returning 50

VII. Screenshots and Evidence

Note: Screenshots for the following have been placed throughout this report next to their matching commands and tasks:

- Terminal showing GCC installation and version check
- Each compilation stage (.i, .s, .o files)
- Program execution with different return values
- Exit status verification using echo \$?

VIII. Conclusion

This lab exercise provided hands-on experience with C programming fundamentals, including program structure, compilation process, and GCC usage. Understanding these concepts is essential for systems programming and operating system development. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions.

GitHub Repository

[Rupeshycho/240565_Rupesh_Yadav_linux](https://github.com/Rupeshycho/240565_Rupesh_Yadav_linux)

The lab reports are placed inside this repository . Please review through this section .